UNIVERSIDAD POLITECNICA DEL BICENTENARIO



REPORTE DE PRACTICAS

Práctica 1: Control de dispositivos básicos

Presenta:

Alejandro Almanza Perez

Vanessa Guadalupe Soria Zavala

Adán Rodriguez Ortiz

America Aranzazu Rodriguez Perez

Matricula:

24033177

24030215

24033563

2403

Asignatura:

Sistemas Embebidos

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto a 11 de septiembre de 2026

INDICE

Introducción 4

Marco Teórico 4

Práctica 1: Control de dispositivos básicos. 8

Practica 1.1 LED 8

Objetivo 8

Material requerido 8

Asignación de pines 8

Desarrollo 9

Diseño del circuito 9

Conexiones 9

Programación 10

Funcionamiento 11

Resultados 12

Observaciones y conclusiones 13

PRÁCTICA 1.2 Motor 14

Objetivo 14

Material requerido 14

Asignación de pines 14

Desarrollo 15

Diseño del circuito 15

Conexiones 16

Programación 17

Funcionamiento 17

Resultados 18

Observaciones y conclusiones 19

PRÁCTICA 1.3 Socket y Foco 20

Objetivo 20

Material requerido 20

Asignación de pines 20

Desarrollo 21

Diseño del circuito 21

Conexiones 22
Programación 23
Funcionamiento 23
Resultados 24
Observaciones y conclusiones 25
Referencias 27

Lista de Figuras

Figura 1. Diseño del circuito LED en wokwi. 9
Figura 2. Circuito encendido de LED en funcionamiento. 12
Figura 3. Circuito físico encendido de LED en funcionamiento. 12
Figura 4. Diseño del circuito Motor en wokwi. 16
Figura 5. Circuito Motor encendido en funcionamiento. 18
Figura 6. Circuito físico del Motor en funcionamiento. 18
Figura 7. Diseño del circuito Socket y Foco. 22
Figura 8. Circuito encendido Socket y Foco en funcionamiento. 24
Figura 9. Circuito físico Socket y Foco en funcionamiento. 25

Introducción

Los sistemas embebidos forman parte esencial de la automatización moderna, ya que permiten que un dispositivo procese información del entorno y responda de manera automática ante distintas condiciones, sin necesidad de intervención humana constante. Su aplicación abarca desde electrodomésticos y dispositivos móviles hasta sistemas de control industrial, donde la precisión y la rapidez de respuesta son fundamentales.

En este reporte se documentan las prácticas realizadas en la asignatura de Sistemas Embebidos, en las cuales se utilizó una tarjeta de desarrollo ESP32 para implementar sistemas de control digital básicos. A través de botones pulsadores como señales de entrada y un LED como señal de salida, se busca comprender de manera práctica cómo un microcontrolador interpreta señales digitales (HIGH y LOW) y las traduce en acciones concretas sobre un circuito físico.

Para el desarrollo de estas prácticas se empleó el entorno de programación Arduino IDE, así como la plataforma de simulación en línea Wokwi, la cual permitió diseñar, programar y probar los circuitos antes de su implementación física en Protoboard. Este enfoque combina teoría y práctica, permitiendo relacionar directamente los conceptos de programación (pinMode, digitalRead, digitalWrite) con su efecto sobre los componentes electrónicos conectados al microcontrolador.

El presente documento incluye el marco teórico necesario para comprender los conceptos utilizados, el desarrollo detallado de cada práctica —incluyendo el diseño del circuito, las conexiones, la programación y el funcionamiento del sistema—, así como los resultados obtenidos y las observaciones y conclusiones derivadas de la experimentación.

Marco Teórico

Los sistemas de automatización industrial requieren dispositivos capaces de procesar información y tomar decisiones a partir de distintas condiciones de entrada. Para comprender el funcionamiento de la práctica desarrollada, es necesario abordar primero los conceptos de sistema embebido, microcontrolador, señales digitales, niveles lógicos y operaciones lógicas básicas.

Sistemas embebidos

Un sistema embebido es un sistema de cómputo diseñado para realizar una función específica, cuyo procesamiento central recae en un microcontrolador o microprocesador integrado junto con memoria e interfaces de entrada/salida en un mismo circuito. A diferencia de una computadora de propósito general, un sistema

embebido se dedica a una tarea concreta, lo cual le permite ejecutarla con mayor precisión, rapidez y menor consumo de recursos.

Estos sistemas combinan hardware y software especializados: el hardware aporta los componentes físicos (sensores, actuadores, memoria e interfaces de comunicación), mientras que el software embebido gestiona su funcionamiento, procesa datos y controla la interacción con el usuario o con otros dispositivos. Entre sus principales ventajas se encuentran el control preciso de procesos, la reducción de costos operativos, la posibilidad de un diseño modular adaptable a distintas necesidades y la capacidad de monitoreo en tiempo real, características que los hacen ideales para aplicaciones de control y automatización industrial.

El microcontrolador ESP32

El ESP32 es un microcontrolador de bajo costo desarrollado por Espressif Systems, que integra conectividad Wi-Fi y Bluetooth de doble modo. Cuenta con un procesador Xtensa de arquitectura de 32 bits (disponible en versiones de uno o dos núcleos) capaz de operar hasta 240 MHz, además de memoria SRAM y ROM integradas.

Entre sus características más relevantes para el desarrollo de proyectos de control se encuentran:

- Hasta 34 pines GPIO (General Purpose Input/Output) configurables como entradas o salidas digitales.
- Dos convertidores analógico-digitales (ADC) de 12 bits, con hasta 18 canales.
- Dos convertidores digital-analógicos (DAC) de 8 bits.
- Múltiples interfaces de comunicación: SPI, I2C, UART e I2S.
- Módulo de PWM con hasta 16 canales, útil para el control de motores y LEDs.

Gracias a la multiplexación de pines, la mayoría de las funciones del ESP32 pueden reasignarse a diferentes GPIO según las necesidades del proyecto. No obstante, existen pines de propósito especial —como los pines de arranque (strapping) o los de entrada únicamente (GPIO 34 a 39)— que deben evitarse o utilizarse con precaución, ya que intervienen en la configuración del microcontrolador durante el encendido.

Entradas y salidas digitales

Los microcontroladores interpretan el mundo físico a través de sus terminales de entrada y salida. Una entrada permite que el microcontrolador reciba una señal de voltaje y, a partir de ella, determine si un evento ha ocurrido (por ejemplo, si un botón fue presionado). Una salida, en cambio, es un terminal mediante el cual el

microcontrolador entrega una señal de voltaje hacia un componente externo, como un LED.

Cuando estas señales solo pueden tomar dos valores discretos, se les denomina señales digitales, en contraposición a las señales analógicas, cuyo voltaje varía de forma continua en función de la magnitud que representan. En el caso de una entrada digital, el microcontrolador únicamente distingue dos estados: HIGH (nivel lógico alto) y LOW (nivel lógico bajo), sin importar variaciones intermedias de voltaje.

Niveles lógicos

Un nivel lógico es la tensión específica o el estado en el que puede encontrarse una señal digital. En electrónica digital se manejan dos estados: encendido (ON) y apagado (OFF), representados en binario como 1 y 0 respectivamente. El estándar más utilizado para definir estos niveles es la lógica TTL (Transistor-Transistor Logic), la cual establece rangos de voltaje aceptables tanto para la entrada como para la salida de un dispositivo digital.

Para un sistema TTL estándar de 5 V se definen los siguientes umbrales:

- **VIH** (tensión mínima de entrada para un nivel alto): aproximadamente 2 V.
- **VIL** (tensión máxima de entrada para un nivel bajo): aproximadamente 0.8 V.
- **VOH** (tensión mínima de salida para un nivel alto): aproximadamente 2.7 V.
- **VOL** (tensión máxima de salida para un nivel bajo): aproximadamente 0.5 V.

La diferencia entre los rangos tolerables de entrada y salida se conoce como margen de ruido, y representa la cantidad de tensión indeseada que un sistema puede tolerar sin interpretar erróneamente una señal. Este concepto es relevante al diseñar circuitos digitales, ya que garantiza que los niveles HIGH y LOW se reconozcan de forma confiable incluso ante pequeñas variaciones de voltaje.

Operación lógica AND

La operación AND requiere que todas las condiciones de entrada sean verdaderas para activar la salida.

Para dos entradas A y B: $\boldsymbol{Y = A * B}$ $\boldsymbol{(1)}$

Su tabla de verdad es:

Tabla 1. Tabla de verdad de la función AND

Entrada A	Entrada B	Salida Y
0	0	0

0	1	0
1	0	0
1	1	1

Operación lógica OR

La operación OR, a diferencia de la AND, activa la salida cuando al menos una de las entradas es verdadera.

Para dos entradas A y B: $\boldsymbol{Y = A + B}$ $\boldsymbol{(2)}$

Su tabla de verdad es:

Tabla 2. Tabla de verdad de la función OR

Entrada A	Entrada B	Salida Y
0	0	0
0	1	1
1	0	1
1	1	1

Control eléctrico y su aplicación en la automatización

El control eléctrico consiste en el uso de dispositivos eléctricos y electrónicos para regular, monitorear y automatizar el funcionamiento de sistemas y máquinas. Puede clasificarse en control manual, cuando requiere la intervención directa de un operador, o control automático, cuando se apoya en sensores, relés y controladores programables (PLC) que ajustan el comportamiento del sistema sin intervención humana constante. Este principio es la base conceptual de la lógica digital aplicada en la presente práctica, donde las entradas (botones) y la salida (LED) simulan, a pequeña escala, el comportamiento de un sistema de control automatizado.

Práctica 1: Control de dispositivos básicos.

Practica 1.1 LED

Objetivo

Implementar un sistema de control digital utilizando una tarjeta ESP32, en el cual dos botones actúen como señales de entrada y un LED como señal de salida, con el fin de comprender de manera práctica cómo el estado de las entradas determina el comportamiento de una salida, aplicando los conceptos de lógica digital, niveles lógicos HIGH/LOW y programación de entradas y salidas en un microcontrolador.

Material requerido

- 1 tarjeta ESP32.
- 2 botones pulsadores.
- 1 resistencia de 1 kΩ.
- 1 LED.
- Protoboard.
- Cables Dupont.
- Cable USB.
- Computadora con Arduino IDE.
- Software Wokwi para simulación.

Asignación de pines

Componente	Pin ESP32	Tipo	Función
LED (ánodo, vía resistencia de 1 kΩ)	GPIO 2	Salida digital	Indicador de encendido/apagado
Push Button 1 (encendido)	GPIO 21	Entrada digital (INPUT_PULLUP)	Enciende el LED al presionarse
Push Button 2 (encendido*)	GPIO 19	Entrada digital (INPUT_PULLUP)	Apaga el LED al presionarse
LED (cátodo)	GND	—	Retorno a tierra
Push Button 1 (terminal 1)	GND	—	Retorno a tierra
Push Button 2 (terminal 1)	GND	—	Retorno a tierra

Desarrollo

Diseño del circuito

El circuito está compuesto por una tarjeta ESP32 DevKit C V4, dos pulsadores (push buttons) y un LED con su respectiva resistencia limitadora de corriente. El LED se conecta al GPIO 2 del ESP32 a través de una resistencia de 1 kΩ en serie, la cual protege al diodo de una corriente excesiva; su cátodo se conecta directamente a GND. Los dos botones se conectan de forma independiente: el primero al GPIO 21 y el segundo al GPIO 19, ambos configurados en modo INPUT_PULLUP, lo que significa que el propio microcontrolador mantiene la entrada en estado HIGH mientras el botón no es presionado, y solo cambia a LOW cuando el circuito se cierra a tierra al pulsarlo. Con este diseño no es necesario agregar resistencias externas de pull-up ni pull-down para los botones, ya que esta función la realiza internamente el ESP32.

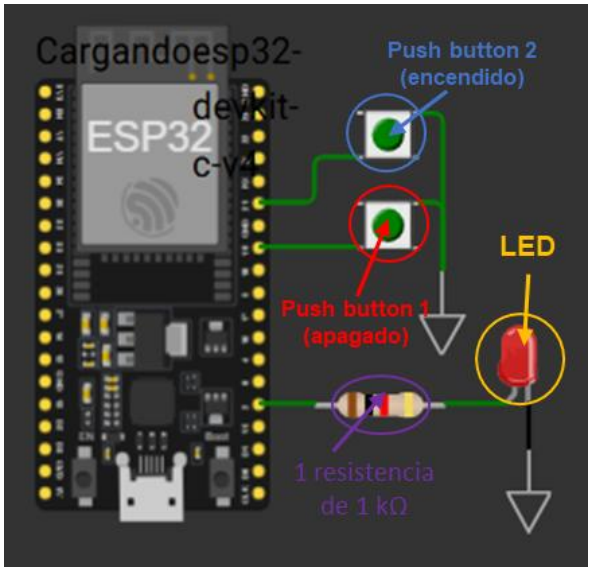


Figura 1. Diseño del circuito LED en wokwi.

Conexiones

- GPIO 2 → Resistencia (1 kΩ) → LED (ánodo) → LED (cátodo) → GND: controla el encendido del LED como salida digital.
- GPIO 21 → Push Button 1 → GND: entrada digital que, al presionarse, enciende el LED.
- GPIO 19 → Push Button 2 → GND: entrada digital que, al presionarse, apaga el LED.

- TX/RX del ESP32 → Monitor Serial: permite visualizar en tiempo real qué botón fue presionado, mediante mensajes impresos por el puerto serial.

Estas conexiones son las que permiten que el microcontrolador detecte el estado físico de cada botón (presionado o no presionado) y reaccione en consecuencia sobre la salida asignada al LED.

Programación

Los códigos usados fueron los siguientes

diagram.json

```
{
  "version": 1,
  "author": "Vanessa Soria Zavala",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left":
0, "attrs": {} },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 157.55,
      "left": 124.8,
      "attrs": { "value": "1000" }
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": 121.2,
      "left": 195.8,
      "attrs": { "color": "red", "flip": "1" }
    },
    { "type": "wokwi-gnd", "id": "gnd1", "top": 182.4, "left": 210.6,
"attrs": {} },
    {
      "type": "wokwi-pushbutton-6mm",
      "id": "btn1",
      "top": 36.2,
      "left": 144,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton-6mm",
      "id": "btn2",
      "top": 74.6,
      "left": 144,
      "attrs": { "color": "green", "xray": "1" }
    },
    { "type": "wokwi-gnd", "id": "gnd2", "top": 105.6, "left": 172.2,
"attrs": {} }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "esp:2", "r1:1", "green", [ "h0" ] ],
    [ "led1:A", "r1:2", "green", [ "v0" ] ],
```

```
    [ "gnd1:GND", "led1:C", "black", [ "v0" ] ],
    [ "esp:19", "btn2:2.l", "green", [ "h0" ] ],
    [ "esp:21", "btn1:2.l", "green", [ "h24.04", "v-19.2" ] ],
    [ "btn1:1.r", "gnd2:GND", "green", [ "v0", "h10.4" ] ],
    [ "btn2:1.r", "gnd2:GND", "green", [ "v0", "h10.4" ] ]
  ],
  "dependencies": {}
}
```

sketch.ino

```
#define LED 2
#define SW1 21
#define SW2 19
#define PRENDE_LED digitalWrite(LED,HIGH)
#define APAGA_LED digitalWrite(LED,LOW)
#define PRESSED(x) digitalRead(x)==0

void setup() {

  Serial.begin(115200);
  Serial.println("Hello, ESP32!");

  pinMode(LED, OUTPUT);
  pinMode(SW1, INPUT_PULLUP);
  pinMode(SW2, INPUT_PULLUP);

}

void loop() {

  if(PRESSED(SW1)){
      PRENDE_LED;
      Serial.println("Boton 1 pulsado");
      delay(100);
  }

 if(PRESSED(SW2)){
      APAGA_LED;
      Serial.println("Boton 2 pulsado");
      delay(100);
  }
}
```

Funcionamiento

1. Al iniciar el programa, se establece comunicación serial a 115200 baudios y se imprime el mensaje "Hello, ESP32!" como confirmación de que la tarjeta arrancó correctamente.
2. Se configuran los pines: GPIO 2 como salida (LED), y GPIO 21 y GPIO 19 como entradas con resistencia de pull-up interna (INPUT_PULLUP).
3. Dentro del ciclo principal (`loop`), el programa evalúa constantemente el estado de ambos botones.

4. Si el Botón 1 (GPIO 21) es presionado, su lectura cambia a LOW (0), lo cual cumple la condición `PRESSED(SW1)`, y el programa enciende el LED (`digitalWrite(LED, HIGH)`), además de imprimir "Boton 1 pulsado" en el monitor serial.
5. Si el Botón 2 (GPIO 19) es presionado, su lectura también cambia a LOW (0), cumpliendo `PRESSED(SW2)`, y el programa apaga el LED (`digitalWrite(LED, LOW)`), imprimiendo "Boton 2 pulsado" en el monitor serial.
6. Este ciclo se repite continuamente, por lo que el LED permanece encendido o apagado según cuál haya sido el último botón presionado.

Resultados

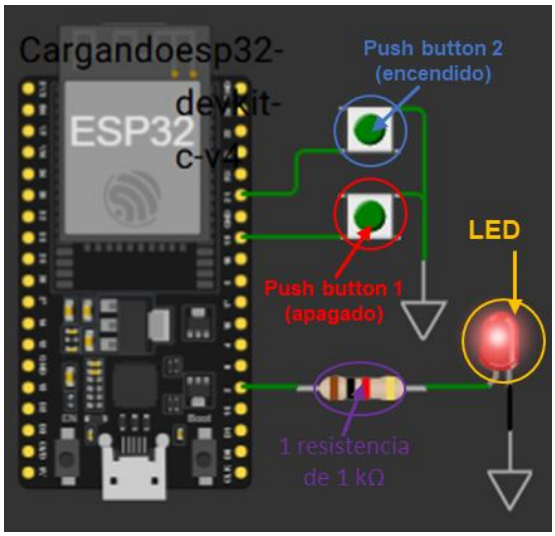


Figura 2. Circuito encendido de LED en funcionamiento.

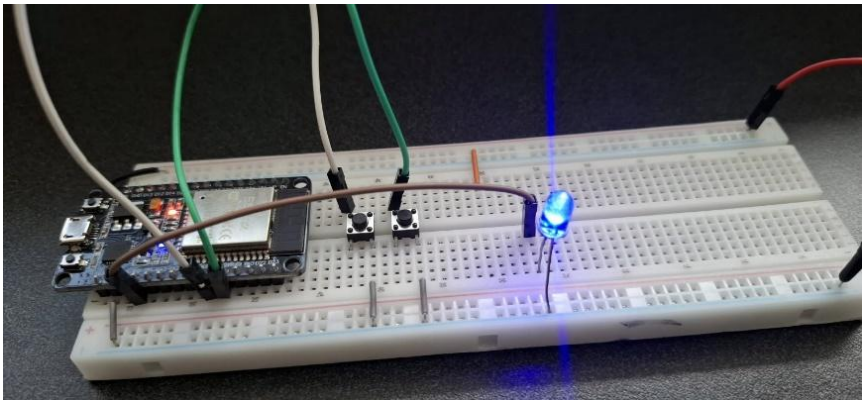


Figura 3. Circuito físico encendido de LED en funcionamiento.

Durante la simulación en Wokwi, el sistema respondió correctamente a las entradas de los dos pulsadores. Al presionar el Botón 1, el LED se encendió de manera inmediata y el monitor serial mostró el mensaje "Boton 1 pulsado". Al presionar el Botón 2, el LED se apagó y el monitor serial mostró "Boton 2 pulsado". El comportamiento se mantuvo estable en todas las pruebas realizadas, confirmando que las lecturas digitales de los botones (HIGH en reposo, LOW al presionarse) se interpretaron correctamente por el microcontrolador y se tradujeron en el cambio de estado esperado en la salida.

Observaciones y conclusiones

Durante el desarrollo de esta práctica se comprendió de forma aplicada cómo un microcontrolador interpreta señales digitales de entrada y las traduce en acciones sobre una salida, en este caso, el encendido o apagado de un LED. Se confirmó la utilidad de la configuración INPUT_PULLUP, ya que evita el uso de resistencias externas y garantiza que la entrada tenga un estado definido (HIGH) cuando el botón no está presionado, evitando lecturas erráticas o flotantes.

No se presentaron problemas durante la implementación ni la simulación del circuito, ya que las conexiones fueron sencillas y el código no requirió ajustes adicionales para funcionar correctamente desde la primera prueba.

Se observa una relación directa entre el programa y el circuito: cada línea de código corresponde a un elemento físico del circuito (`pinMode`, `digitalRead` y `digitalWrite` reflejan exactamente cómo están conectados el LED y los botones a los GPIO del ESP32). Esto evidencia que la lógica de programación no es independiente del hardware, sino que debe diseñarse en función de cómo están físicamente conectados los componentes.

Finalmente, esta práctica permitió comprobar de manera experimental los conceptos vistos en el marco teórico, particularmente el manejo de niveles lógicos HIGH/LOW y el uso de entradas y salidas digitales en un microcontrolador, sentando una base sólida para futuras prácticas donde se implementen operaciones lógicas más complejas, como AND y OR, combinando el estado de ambos botones para activar una misma salida.

PRÁCTICA 1.2 Motor

Objetivo

Implementar un sistema de control digital utilizando una tarjeta ESP32, en el cual dos botones actúen como señales de entrada y un motor de corriente directa (DC) como señal de salida, controlado a través de un transistor NPN como interruptor electrónico. Con ello se busca comprender cómo un microcontrolador puede controlar cargas que exceden su capacidad de corriente, aplicando los conceptos de conmutación con transistores, niveles lógicos HIGH/LOW y alimentación externa de un circuito.

Material requerido

- 1 tarjeta ESP32.
- 2 botones pulsadores.
- 1 motor de corriente directa (DC) de bajo voltaje.
- 1 transistor NPN 2N2222.
- 1 diodo de protección (flyback) 1N4007 o 1N4001.
- 1 resistencia de 1 kΩ (base del transistor).
- 1 fuente de alimentación externa de 5 V.
- Protoboard.
- Cables Dupont.
- Cable USB.
- Computadora con Arduino IDE.

Asignación de pines

Componente	Pin ESP32	Tipo	Función
Base del transistor (vía resistencia de 1 kΩ)	GPIO 2	Salida digital	Controla la conmutación del transistor (enciende/apaga el motor)
Push Button 1 (encendido)	GPIO 21	Entrada digital (INPUT_PULLUP)	Enciende el motor al presionarse
Push Button 2 (apagado)	GPIO 19	Entrada digital (INPUT_PULLUP)	Apaga el motor al presionarse

Emisor del transistor	GND (ESP32)	—	Retorno a tierra común
Push Button 1 (terminal 1)	GND	—	Retorno a tierra
Push Button 2 (terminal 1)	GND	—	Retorno a tierra
Motor (terminal +)	+5 V (fuente externa)	—	Alimentación del motor
Motor (terminal −) / Colector del transistor	Colector del transistor	—	Controla el paso de corriente al motor
Diodo flyback (cátodo)	+5 V (fuente externa)	—	Protección contra corriente inversa
Diodo flyback (ánodo)	Colector del transistor	—	Protección contra corriente inversa

Desarrollo

Diseño del circuito

A diferencia del LED, un motor DC requiere corriente muy superior a la que un GPIO del ESP32 puede entregar (máximo ~40 mA), por lo que no puede conectarse directamente al microcontrolador. Para resolver esto se utiliza un transistor NPN funcionando como interruptor electrónico: el GPIO 2 controla la base del transistor a través de una resistencia de 1 kΩ (que limita la corriente de base), mientras que el colector y el emisor controlan el paso de la corriente principal que alimenta al motor.

El motor se alimenta desde una fuente externa de 5 V, independiente del voltaje que entrega el ESP32 por USB, ya que el consumo de un motor puede sobrepasar la capacidad de la tarjeta. Es indispensable unir el GND de la fuente externa con el GND del ESP32, de modo que ambos circuitos compartan una referencia de tierra común; de lo contrario, el transistor no podría conmutar correctamente.

Adicionalmente, se coloca un diodo de protección (flyback) en paralelo con el motor, con el cátodo hacia la alimentación positiva y el ánodo hacia el colector del transistor. Este diodo es fundamental porque los motores, al ser cargas inductivas, generan un pico de voltaje inverso (fuerza contraelectromotriz) en el instante en que se cortan la corriente; sin el diodo, este pico podría dañar el transistor.

Los dos botones se mantienen conectados exactamente igual que en la Práctica 1: el primero al GPIO 21 y el segundo al GPIO 19, ambos en modo INPUT_PULLUP.

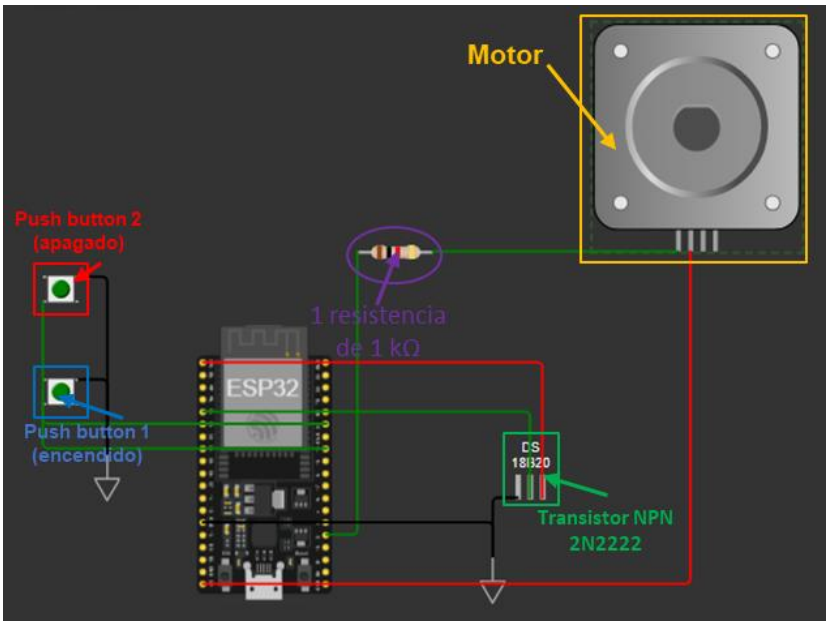


Figura 4. Diseño del circuito Motor en wokwi.

Conexiones

- GPIO 2 → Resistencia (1 kΩ) → Base del transistor: señal de control que activa o desactiva la conmutación.
- Emisor del transistor → GND (común con ESP32): referencia de tierra compartida.
- Colector del transistor → Terminal (−) del motor: controla el paso de corriente hacia el motor.
- Terminal (+) del motor → +5 V (fuente externa): alimentación principal del motor.
- Diodo flyback en paralelo al motor (cátodo a +5 V, ánodo al colector): protección contra picos de voltaje inverso.
- GPIO 21 → Push Button 1 → GND: entrada digital que, al presionarse, enciende el motor.
- GPIO 19 → Push Button 2 → GND: entrada digital que, al presionarse, apaga el motor.

Estas conexiones permiten que el ESP32 controle, de forma segura y aislada, una carga externa de mayor consumo que la que sus propios pines pueden manejar.

Programación

Código utilizado (Arduino IDE)

```
#define MOTOR 2
#define SW1 21
#define SW2 19
#define ENCIENDE_MOTOR digitalWrite(MOTOR, HIGH)
#define APAGA_MOTOR digitalWrite(MOTOR, LOW)
#define PRESSED(x) digitalRead(x) == 0

void setup() {

  Serial.begin(115200);
  Serial.println("Hello, ESP32!");

  pinMode(MOTOR, OUTPUT);
  pinMode(SW1, INPUT_PULLUP);
  pinMode(SW2, INPUT_PULLUP);

}

void loop() {

  if (PRESSED(SW1)) {
    ENCIENDE_MOTOR;
    Serial.println("Boton 1 pulsado: Motor encendido");
    delay(100);
  }

  if (PRESSED(SW2)) {
    APAGA_MOTOR;
    Serial.println("Boton 2 pulsado: Motor apagado");
    delay(100);
  }
}
```

Funcionamiento

1. Al iniciar el programa, se establece comunicación serial a 115200 baudios y se imprime "Hello, ESP32!" como confirmación de arranque.
2. Se configuran los pines: GPIO 2 como salida (control del transistor), y GPIO 21 y GPIO 19 como entradas con resistencia de pull-up interna.
3. Dentro del ciclo principal (`loop`), el programa evalúa constantemente el estado de ambos botones.
4. Si el Botón 1 (GPIO 21) es presionado, su lectura cambia a LOW, cumpliendo `PRESSED(SW1)`; el GPIO 2 pasa a HIGH, lo cual satura el transistor y permite el paso de corriente desde la fuente de 5 V hacia el motor, encendiéndolo.
5. Si el Botón 2 (GPIO 19) es presionado, su lectura cambia a LOW, cumpliendo `PRESSED(SW2)`; el GPIO 2 pasa a LOW, el transistor deja de conducir y el motor se apaga.

6. El diodo flyback absorbe el pico de voltaje generado cada vez que el motor se apaga, protegiendo al transistor de daños.
7. Este ciclo se repite continuamente, por lo que el motor permanece encendido o apagado según cuál haya sido el último botón presionado.

Resultados

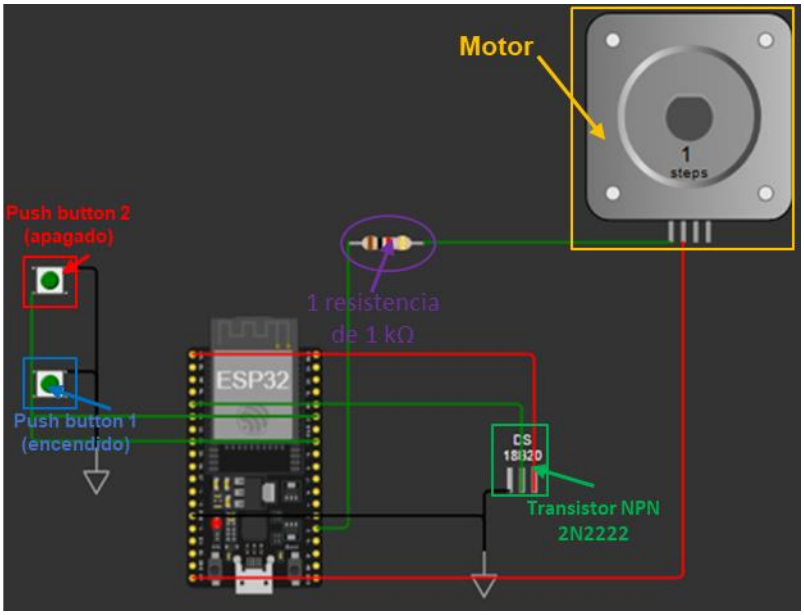


Figura 5. Circuito Motor encendido en funcionamiento.

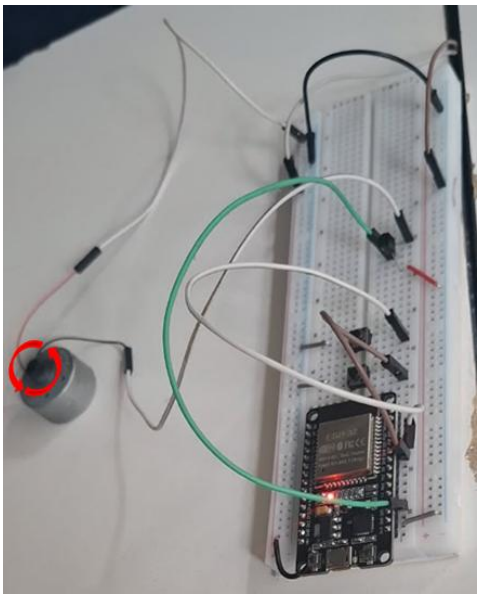


Figura 6. Circuito físico del Motor en funcionamiento.

Al presionar el Botón 1, el transistor conmutó correctamente y el motor comenzó a girar de manera inmediata, mostrando en el monitor serial el mensaje "Boton 1 pulsado: Motor encendido". Al presionar el Botón 2, el motor se detuvo por completo y el monitor serial mostró "Boton 2 pulsado: Motor apagado". El comportamiento se mantuvo estable durante las pruebas, confirmando que el transistor funcionó correctamente como interruptor electrónico controlado por el ESP32, y que la fuente externa de 5 V suministró la corriente necesaria sin afectar el funcionamiento del microcontrolador.

Observaciones y conclusiones

Esta práctica permitió comprender que, a diferencia de una carga de bajo consumo como un LED, un motor requiere de un componente intermedio —en este caso, un transistor NPN— para poder ser controlado por un microcontrolador, ya que los GPIO del ESP32 no están diseñados para entregar corrientes elevadas. Se comprobó experimentalmente el principio de conmutación de un transistor: una señal de baja corriente en la base (proveniente del GPIO) es capaz de controlar una corriente mucho mayor entre el colector y el emisor, permitiendo así aislar la parte lógica del circuito (3.3 V del ESP32) de la parte de potencia (5 V de la fuente externa).

Asimismo, se identificó la importancia del diodo de protección (flyback), ya que los motores son cargas inductivas que generan picos de voltaje inverso al interrumpir la corriente; sin este diodo, dichos picos podrían dañar el transistor a mediano o largo plazo, incluso si el circuito parece funcionar bien en pruebas cortas.

Un aspecto crítico durante el diseño fue asegurar que la fuente externa de 5 V y el ESP32 compartieran una misma referencia de tierra (GND común); sin esta conexión, el transistor no habría podido conmutar de forma confiable, ya que la señal de control del GPIO no tendría una referencia válida respecto al circuito de potencia.

Finalmente, se observa que, aunque el software de esta práctica es prácticamente idéntico al de la Práctica 1 (mismas variables lógicas HIGH/LOW y misma estructura de código), la diferencia sustancial se encuentra en el diseño del hardware, lo que confirma que el comportamiento de un sistema embebido depende tanto de la programación como del diseño correcto del circuito electrónico que interactúa con el mundo físico.

PRÁCTICA 1.3 Socket y Foco

Objetivo

Implementar un sistema de control digital utilizando una tarjeta ESP32, en el cual dos botones actúen como señales de entrada y un foco conectado a un socket de corriente alterna (110-127 V AC) como señal de salida, controlado por medio de un relevador (relay). Con ello se busca comprender cómo un microcontrolador puede controlar de forma segura una carga de alto voltaje, mediante el aislamiento eléctrico que proporciona un relevador entre el circuito de control (bajo voltaje) y el circuito de potencia (alto voltaje).

Material requerido

- 1 tarjeta ESP32.
- 2 botones pulsadores.
- 1 módulo relevador (relay) de 5 V, 2 canales.
- 1 socket para foco.
- 1 foco incandescente o LED de 110-127 V AC.
- Cable de alimentación AC (clavija).
- Protoboard.
- Cables Dupont.
- Cable USB.
- Computadora con Arduino IDE.
- 1 fuente de alimentación externa de 5 V.

Asignación de pines

Componente	Pin ESP32	Tipo	Función
Módulo relevador (pin IN / señal de control)	GPIO 2	Salida digital	Activa/desactiva la bobina del relevador
Push Button 1 (encendido)	GPIO 21	Entrada digital (INPUT_PULLUP)	Enciende el foco al presionarse
Push Button 2 (apagado)	GPIO 19	Entrada digital (INPUT_PULLUP)	Apaga el foco al presionarse
Módulo relevador (VCC)	5 V (ESP32 o fuente externa)	—	Alimentación de la bobina del relevador
Módulo relevador (GND)	GND	—	Retorno a tierra común
Push Button 1 (terminal 1)	GND	—	Retorno a tierra

Push Button 2 (terminal 1)	GND	—	Retorno a tierra
Socket (terminal 1)	Línea AC (fase)	—	Alimentación de corriente alterna
Socket (terminal 2)	Relevador (contacto COM/NO)	—	Corriente controlada hacia el foco

Nota de seguridad: el lado de contactos del relevador (COM, NO, NC) maneja voltaje de línea AC (110-127 V), el cual puede ser peligroso. Estas conexiones deben aislarse físicamente del circuito de control de bajo voltaje (ESP32, botones) y manipularse con la corriente desconectada.

Desarrollo

Diseño del circuito

Al tratarse de un foco alimentado con corriente alterna de línea (110-127 V), no es posible controlarlo directamente ni con un GPIO ni con un transistor como en la práctica del motor, ya que estos manejan corriente directa (DC) a bajo voltaje. Para esta práctica se utiliza un **módulo relevador (relay)**, el cual actúa como un interruptor mecánico controlado eléctricamente: su bobina interna se energiza con una señal de bajo voltaje (proveniente del GPIO del ESP32), lo que provoca el cierre (o apertura) de un contacto mecánico que sí puede manejar la corriente AC del foco.

Esta configuración proporciona **aislamiento eléctrico** entre el circuito de control (3.3 V/5 V, ESP32) y el circuito de potencia (110-127 V AC, foco), lo cual es fundamental por seguridad: cualquier falla en el lado de alto voltaje no debe poder regresar hacia el microcontrolador.

El pin de señal (IN) del módulo relevador se conecta al GPIO 2 del ESP32; la alimentación del módulo (VCC y GND) se conecta a los pines de 5 V y GND de la tarjeta. Del lado de potencia, la línea de fase (AC) se conecta a uno de los contactos del relevador (normalmente abierto, NO), y el otro contacto se conecta al socket del foco, cerrando así el circuito solo cuando el relevador es activado.

Los dos botones se conectan de la misma manera que en las prácticas anteriores: el primero al GPIO 21 y el segundo al GPIO 19, ambos en modo INPUT_PULLUP.

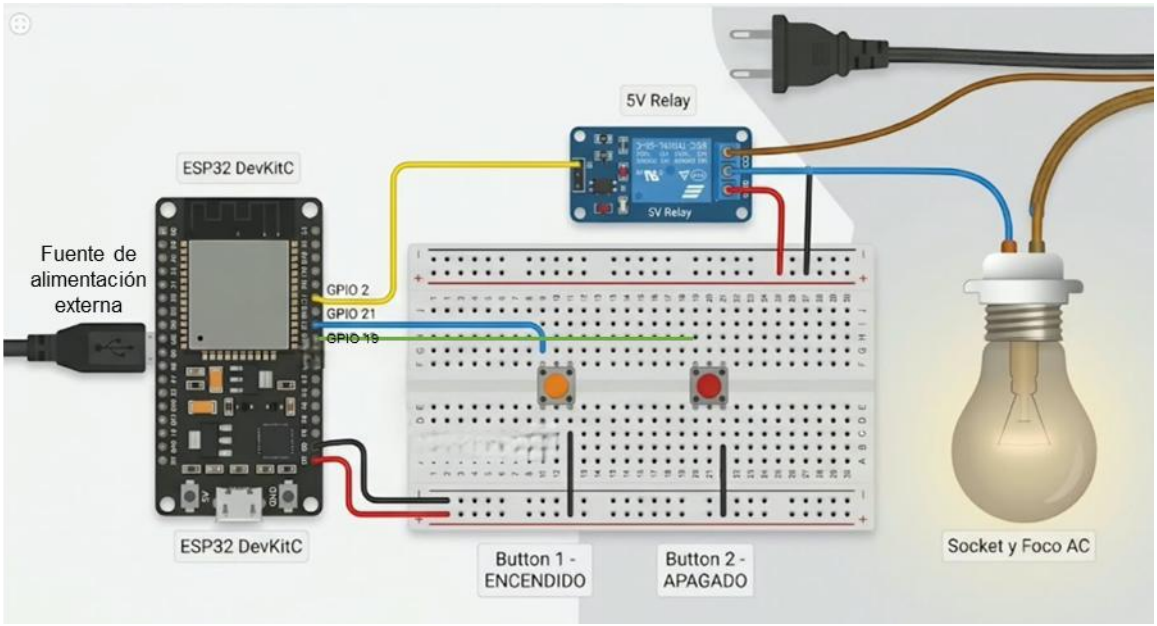


Figura 7. Diseño del circuito Socket y Foco.

Conexiones

- GPIO 2 → Pin IN del módulo relevador: señal de control que activa o desactiva la bobina del relevador.
- VCC del relevador → 5 V del ESP32: alimentación del módulo.
- GND del relevador → GND del ESP32: referencia de tierra común.
- Línea de fase (AC) → Contacto COM del relevador: entrada de corriente alterna.
- Contacto NO del relevador → Socket del foco: corriente controlada hacia el foco (el circuito solo se cierra cuando el relevador está activado).
- GPIO 21 → Push Button 1 → GND: entrada digital que, al presionarse, activa el relevador y enciende el foco.
- GPIO 19 → Push Button 2 → GND: entrada digital que, al presionarse, desactiva el relevador y apaga el foco.

Estas conexiones permiten controlar una carga de alto voltaje desde un microcontrolador de bajo voltaje, manteniendo ambos circuitos eléctricamente aislados por medio del relevador.

Programación

Código utilizado (Arduino IDE):

```
#define RELAY 2
#define SW1 21
#define SW2 19
#define ENCIENDE_FOCO digitalWrite(RELAY, HIGH)
#define APAGA_FOCO digitalWrite(RELAY, LOW)
#define PRESSED(x) digitalRead(x) == 0

void setup() {

  Serial.begin(115200);
  Serial.println("Hello, ESP32!");

  pinMode(RELAY, OUTPUT);
  pinMode(SW1, INPUT_PULLUP);
  pinMode(SW2, INPUT_PULLUP);

  APAGA_FOCO; // Estado inicial: foco apagado

}

void loop() {

  if (PRESSED(SW1)) {
    ENCIENDE_FOCO;
    Serial.println("Boton 1 pulsado: Foco encendido");
    delay(100);
  }

  if (PRESSED(SW2)) {
    APAGA_FOCO;
    Serial.println("Boton 2 pulsado: Foco apagado");
    delay(100);
  }
}
```

Nota: al igual que en la práctica del motor, la lógica de programación es idéntica a la Práctica 1 (LED): el GPIO 2 simplemente cambia entre HIGH y LOW. Lo que varía entre prácticas es exclusivamente el hardware conectado a esa salida (LED, transistor+motor, o relevador+foco). Se añadió la línea `APAGA_FOCO` en el `setup()` para asegurar que el foco inicie apagado y no en un estado indefinido al energizar el ESP32.

Funcionamiento

1. Al iniciar el programa, se establece comunicación serial a 115200 baudios y se imprime "Hello, ESP32!" como confirmación de arranque.

2. Se configura el GPIO 2 como salida (control del relevador), y GPIO 21 y GPIO 19 como entradas con resistencia de pull-up interna. Además, se fuerza el estado inicial del relevador a LOW (foco apagado).
3. Dentro del ciclo principal (`loop`), el programa evalúa constantemente el estado de ambos botones.
4. Si el Botón 1 (GPIO 21) es presionado, su lectura cambia a LOW, cumpliendo `PRESSED(SW1)`; el GPIO 2 pasa a HIGH, energizando la bobina del relevador, lo cual cierra el contacto NO y permite el paso de corriente AC hacia el foco, encendiéndolo.
5. Si el Botón 2 (GPIO 19) es presionado, su lectura cambia a LOW, cumpliendo `PRESSED(SW2)`; el GPIO 2 pasa a LOW, desenergizando la bobina, lo cual abre el contacto y corta la corriente hacia el foco, apagándolo.
6. Este ciclo se repite continuamente, por lo que el foco permanece encendido o apagado según cuál haya sido el último botón presionado.

Resultados

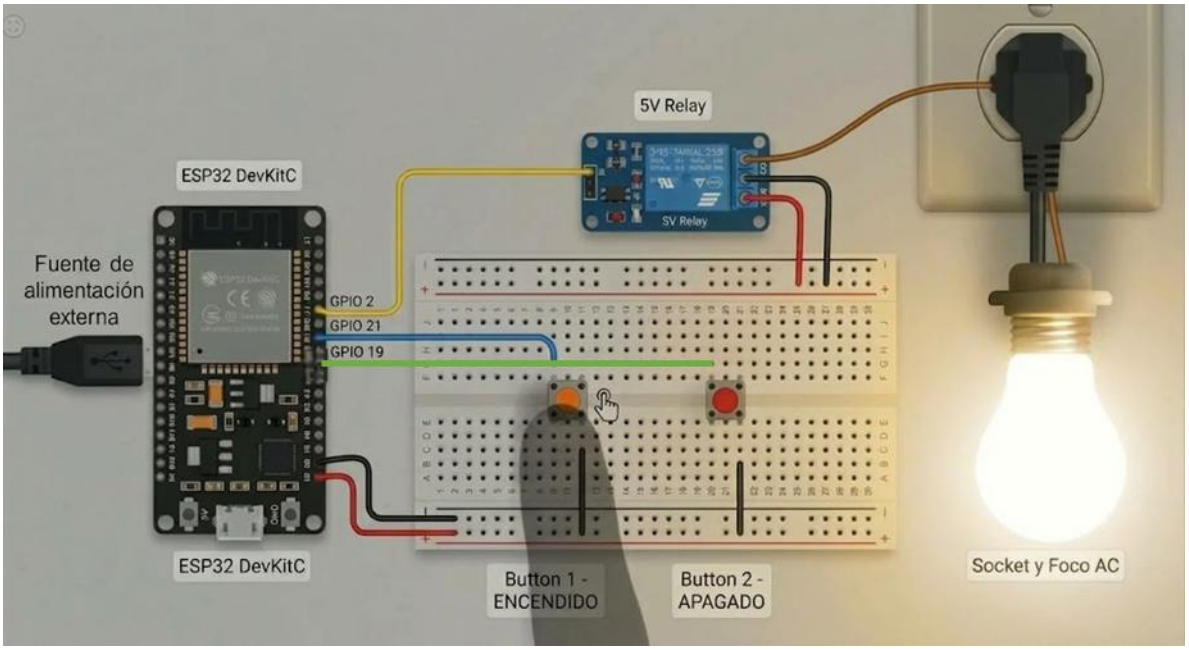


Figura 8. Circuito encendido Socket y Foco en funcionamiento.

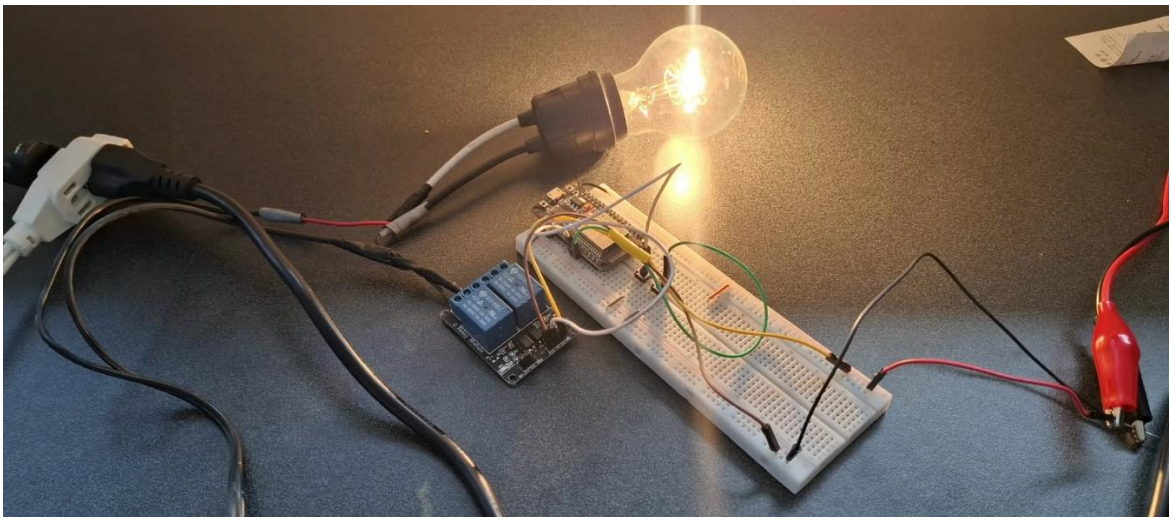


Figura 9. Circuito físico Socket y Foco en funcionamiento.

Al presionar el Botón 1, el relevador cambió de estado de forma audible (clic característico de la bobina) y el foco encendió de manera inmediata, mostrando en el monitor serial el mensaje "Boton 1 pulsado: Foco encendido". Al presionar el Botón 2, el relevador regresó a su posición de reposo y el foco se apagó, mostrando "Boton 2 pulsado: Foco apagado". El comportamiento se mantuvo estable en todas las pruebas, confirmando que el relevador respondió correctamente a las señales digitales del ESP32 y que el aislamiento entre el circuito de control y el circuito de potencia funcionó como se esperaba.

Observaciones y conclusiones

Esta práctica permitió comprender la diferencia fundamental entre controlar cargas de baja potencia en corriente directa (como el LED o el motor de las prácticas anteriores) y controlar cargas de corriente alterna de línea, como un foco. Mientras que el transistor permite controlar cargas DC de forma directa mediante conmutación electrónica, el manejo de corriente alterna de alto voltaje requiere un dispositivo de aislamiento físico, como el relevador, que separa completamente el circuito de control del circuito de potencia mediante un contacto mecánico.

Se comprobó experimentalmente el principio de funcionamiento de un relevador: una señal de bajo voltaje (3.3 V/5 V) proveniente del GPIO es capaz de energizar una bobina electromagnética, la cual a su vez mueve un contacto mecánico capaz de manejar corrientes y voltajes mucho mayores (en este caso, 110-127 V AC), sin que exista una conexión eléctrica directa entre ambos circuitos.

Asimismo, se reforzó la importancia de la seguridad eléctrica al trabajar con corriente alterna de línea: las conexiones del lado de alto voltaje (fase, contactos del relevador, socket) deben mantenerse completamente aisladas del área de trabajo de bajo voltaje, y toda manipulación del circuito debe realizarse con la alimentación AC desconectada, ya que se tuvieron complicaciones menores tal como lo fue el conectar de manera adecuada ya que aunque el foco estuvo bien no logramos al primer intento el conectar bien.

Finalmente, se observa nuevamente que el código de programación no cambió sustancialmente respecto a las prácticas anteriores (las mismas variables HIGH/LOW controlando una salida digital), lo que confirma que, en sistemas embebidos, la complejidad de una aplicación recae frecuentemente en el diseño del hardware de potencia y en las consideraciones de seguridad, más que en la lógica de programación en sí misma.

Referencias

[1] TRBL Services. (s. f.). *Sistema embebido y sus características: Conceptos fundamentales*. TRBL Services. https://trbl-services.eu/blog-sistema-embebido-caracteristicas/

[2] SD Industrial. (s. f.). *Sistemas embebidos*. SD Industrial. https://sdindustrial.com.mx/blog/sistemas-embebidos/

[3] Llamas, L. (s. f.). *Pinout y detalles del hardware del ESP32*. Luis Llamas. https://www.luisllamas.es/esp32-detalles-hardware-pinout/

[4] Acebott. (s. f.). *Introducción al ESP32*. Acebott. https://acebott.com/es/docs/getting-started-with-esp32/

[5] Panama Hitek. (s. f.). *Entradas y salidas analógicas y digitales*. Panama Hitek. https://panamahitek.com/entradas-y-salidas-analogicas-y-digitales/

[6] MCI Electronics. (2022). *Niveles lógicos*. MCI Electronics. https://cursos.mcielectronics.cl/2022/08/05/niveles-logicos/

[7] All About Circuits. (s. f.). *Niveles de voltaje de señales lógicas*. All About Circuits. https://www.allaboutcircuits.com/textbook/digital/chpt-3/logic-signal-voltage-levels/

[8] Datatechnic. (s. f.). *Control eléctrico: Fundamentos y aplicaciones en la industria moderna*. Datatechnic. https://datatechnic.com.mx/control-electrico-conceptos-basicos-y-aplicaciones-en-la-industria/